

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Escuela de Ciencias y Sistemas
Área de desarrollo de software
Software Avanzado

Lic. Marco Tulio Aldana Prillwitz
Xhunik Nikol Miguel Mutzutz



Practica 1

Índice

1. Resumen	3
2. Objetivos	3
2.1 Objetivo General	3
2.2 Objetivos Específicos	3
3. Enunciado	3
3.1 Definiciones	3
3.2 Versión Funcional	4
3.2.1 Modelo de datos mínimo (reglas de validez incluidas)	4
3.2.2 Reglas de cálculo (obligatorias y exactas)	5
3.2.3 Regla de cancelación (obligatoria)	6
3.2.4 Persistencia (obligatoria)	6
3.2.5 Operaciones gRPC (obligatorias)	6
3.3 Versión Final	7
3.3.1 Requisitos SOLID (obligatorio y verificable)	7
3.3.2 Unit Testing (obligatorio)	7
3.3.3 Contrato gRPC y colección Postman (obligatorios)	8
3.3.4 Manejo de errores gRPC (obligatorio)	8
3.3.5 Evidencia por tags (obligatorio)	9
4. Entregables	9
5. Requerimientos mínimos	9
6. Restricciones	10
7. Consideraciones adicionales	10
7.1 Historial de desarrollo	10
7.2 Cálculo de nota individual	10
7.3 Integridad académica	11
8. Fecha y Modo de entrega	11

1. Resumen

En esta práctica desarrollarás un microservicio llamado **QuetzalShip** que permite **crear, consultar, listar, cancelar** órdenes de envío y **generar un recibo** con el **desglose del cálculo tarifario**. El servicio debe exponerse **exclusivamente por gRPC** y debe estar implementado **obligatoriamente con NestJS + TypeScript**.

La práctica busca que el proyecto sea **modular, extensible y testable**, evidenciando claramente los principios **SOLID** (SRP, OCP, ISP, DIP), acompañado de **pruebas unitarias** y un manejo consistente de **errores** y **casos bordes**. La verificación de funcionamiento se realizará con el **archivo .proto**, una **colección de Postman (gRPC)** y la ejecución de **unit tests**.

2. Objetivos

2.1 Objetivo General

Construir un servicio gRPC funcional en NestJS + TypeScript y estructurarlo para obtener una solución **modular, extensible y testable**, respaldada por **pruebas unitarias**, documentación reproducible y manejo consistente de errores.

2.2 Objetivos Específicos

1. Implementar el microservicio con todas las operaciones solicitadas y reglas de negocio correctas.
2. Aplicar SOLID de forma verificable mediante la separación de responsabilidades, el uso de abstracciones y un diseño extensible.
3. Cubrir reglas de cálculo, validaciones y casos bordes con pruebas unitarias.
4. Entregar el contrato gRPC (**.proto**) y una colección Postman (gRPC) para ejecutar todas las llamadas con ejemplos válidos e inválidos.
5. Documentar instalación, ejecución, pruebas, supuestos y etiquetado del repositorio en un README claro.

3. Enunciado

3.1 Definiciones

Término	Definición
Zona	METRO, INTERIOR, FRONTERA
Tipo de servicio	STANDARD, EXPRESS, SAME_DAY

Estado	ACTIVE, CANCELLED
Orden	Conjunto de paquetes con origen, destino y tipo de servicio común, más descuentos/opciones
Paquete	Unidad física enviada dentro de una orden
Desglose	Valores intermedios del cálculo: pesos, subtotales, recargos, descuento y total
Recibo	Representación estable del detalle + desglose, consumible por clientes

3.2 Versión Funcional

El tag **P1-LEGACY** debe apuntar a un commit donde el servicio:

1. ya expone los RPC requeridos por gRPC,
2. valida entradas,
3. calcula tarifas correctamente,
4. permite consultar/listar/cancelar/recibo,
5. mantiene órdenes en memoria durante la ejecución.

3.2.1 Modelo de datos mínimo (reglas de validez incluidas)

Orden

- **orderId**: string único **generado por el servidor**.
- **createdAt**: timestamp **generado por el servidor**.
- **originZone**: Zona.
- **destinationZone**: Zona.
- **serviceType**: Tipo de servicio.
- **packages**: lista **no vacía** con 1 o más paquetes.
- **discount** (opcional):
 - **type**: NONE | PERCENT | FIXED
 - **value**: número (interpretación depende del tipo)
- **insuranceEnabled**: boolean.
- **status**: Estado.

Paquete

- **weightKg**: número **mayor que 0**.
- **heightCm, widthCm, lengthCm**: números **mayores que 0**.
fragile: boolean.
- **declaredValueQ**: número **mayor o igual que 0**.

Regla de validación del seguro

- Si `insuranceEnabled = true`, entonces `sum(declaredValueQ)` de todos los paquetes debe ser **mayor que 0**. Si no se cumple, la solicitud es inválida.

3.2.2 Reglas de cálculo (obligatorias y exactas)

El cálculo debe ser determinista y producir siempre el mismo resultado con la misma entrada.

1. **Peso volumétrico por paquete**
`volumetricKg = (heightCm * widthCm * lengthCm) / 5000`
2. **Peso tarifiable por paquete**
`billableKg = max(weightKg, volumetricKg)`
3. **Peso tarifiable total de la orden**
`orderBillableKg = suma(billableKg de todos los paquetes)`
4. **Tarifa base por kg (según zona destino)**
 - METRO: 8 quetzales por kg
 - INTERIOR: 12 quetzales por kg
 - FRONTERA: 16 quetzales por kg
5. **Subtotal base**
`baseSubtotal = orderBillableKg * rate(destinationZone)`
6. **Multiplicador por tipo de servicio**
 - STANDARD: 1.00
 - EXPRESS: 1.35
 - SAME_DAY: 1.80

`serviceSubtotal = baseSubtotal * multiplier(serviceType)`

7. **Recargos**
 - **Frágil:** 7 quetzales por cada paquete con `fragile = true`.
 - **Seguro:** si `insuranceEnabled = true`, recargo = `0.025 * sum(declaredValueQ)`.

`subtotalWithSurcharges = serviceSubtotal + fragileSurcharge + insuranceSurcharge`

8. **Descuento** (aplica sobre `subtotalWithSurcharges`)
 - **PERCENT:** descuento = `value/100 * subtotalWithSurcharges`, con **tope** `value ≤ 35`. Si `value > 35`, la solicitud es inválida.
 - **FIXED:** descuento = `value` quetzales. El total no puede ser negativo: si `value` excede el subtotal, el total final queda en **0.00**.
9. **Redondeo**
El total final debe redondearse a **2 decimales**.
Para evitar ambigüedades de punto flotante, se recomienda representar dinero

internamente en **centavos** (enteros). Si se usa otra estrategia, debe quedar documentada en README.

3.2.3 Regla de cancelación (obligatoria)

Cancelar una orden cambia su estado a **CANCELLED**.

Una orden cancelada:

- no se elimina,
- conserva el detalle y desglose,
- mantiene su total inmutable.

Cancelar una orden ya cancelada debe responder como error controlado (ver 3.3.4).

3.2.4 Persistencia (obligatoria)

- La persistencia es **en memoria**.
- Reiniciar el servicio borra datos (no hay base de datos).
- El **orderId** debe permanecer único durante el ciclo de ejecución.

3.2.5 Operaciones gRPC (obligatorias)

El servicio gRPC debe incluir **al menos** estos RPC (nombres exactos sugeridos; pueden variar si el **.proto** es coherente y completo):

RPC	Propósito	Respuesta mínima esperada
CreateOrder	Crear orden, validar y calcular	orderId , status , createdAt , desglosado y total
ListOrders	Listar órdenes en resumen	lista de: orderId , destinationZone , serviceType , status , total
GetOrder	Obtener detalle completo	orden completa + desglose
CancelOrder	Cancelar y congelar	orden actualizada + total/desglose
GetReceipt	Generar recibo estable	recibo (detalle + desglose + metadatos)

Requisitos de comportamiento:

- **ListOrders** debe incluir órdenes **ACTIVE** y **CANCELLED**. Si implementas paginación, debe tener defaults explícitos (por ejemplo, **pageSize** por defecto y límites máximos).

- `GetReceipt` debe corresponder exactamente a lo calculado/almacenado para esa orden, respetando congelamiento en cancelación.

3.3 Versión Final

El tag `P1-REFACTOR` debe apuntar al commit final de entrega. Debe mantener el comportamiento funcional de la versión funcional, pero cumplir, además, con las exigencias de diseño, pruebas y consumo indicadas a continuación.

3.3.1 Requisitos SOLID (obligatorio y verificable)

La estructura del proyecto debe permitir identificar claramente, por código y dependencias, lo siguiente:

- **SRP:** el transporte gRPC no debe contener lógica de negocio “pesada”; la validación, el cálculo, el almacenamiento y los casos de uso deben estar separados.
- **OCP:** agregar un nuevo tipo de servicio o una nueva regla de recargo debe ser posible incorporando nuevas clases/estrategias/handlers sin reescribir una lógica central monolítica.
- **ISP:** interfaces pequeñas y específicas (p. ej., repositorio de órdenes ≠ calculadora ≠ generador de recibo).
- **DIP:** casos de uso dependen de abstracciones (interfaces/tokens), no de implementaciones concretas.

Requisito práctico: el cálculo y validaciones principales deben poder probarse sin levantar el servidor gRPC.

3.3.2 Unit Testing (obligatorio)

Debe existir un conjunto mínimo de pruebas unitarias con Jest que cubra, como mínimo:

Cálculo

- volumétrico por paquete,
- tarifiable por paquete,
- acumulación multi-paquete,
- tarifa base por zona destino,
- multiplicador por servicio,
- recargo frágil por paquete,
- recargo por seguro,
- descuento PERCENT con límite 35,
- descuento FIXED con truncamiento a 0.00,
- redondeo final a 2 decimales.

Validaciones y casos borde

- peso o dimensiones menores o iguales a 0,
- `insuranceEnabled=true` con suma declarada igual a 0,

- descuento PERCENT con value mayor a 35,
- orderId inexistente en `GetOrder`, `CancelOrder` y `GetReceipt`,
- doble cancelación.

Los unit tests deben ejecutarse con un comando único:

- `npm test`

3.3.3 Contrato gRPC y colección Postman (obligatorios)

Protofile

- Debe existir `proto/quetzalship.proto` con todos los mensajes y el servicio completo.
- Debe usar `google.protobuf.Timestamp` para fechas.
- Debe representar dinero de forma consistente y sin ambigüedad (por ejemplo, centavos como entero o string normalizada). La decisión debe quedar documentada en README y reflejada en el `.proto`.

Postman (gRPC)

- Debe existir una colección exportada en JSON en:
 - `docs/postman/QuetzalShip.postman_collection.json`
- Debe incluir las 5 operaciones obligatorias con:
 - **2 casos válidos por operación**
 - **1 caso inválido por operación** (para observar el error y su código)

3.3.4 Manejo de errores gRPC (obligatorio)

El servicio debe usar códigos gRPC coherentes para los principales escenarios:

Escenario	Código gRPC
Validación fallida (dimensiones/peso inválidos, seguro inválido, descuento PERCENT > 35, request incompleto)	<code>INVALID_ARGUMENT</code>
<code>orderId</code> no existe	<code>NOT_FOUND</code>
Operación inválida por estado (ej. cancelar ya cancelada)	<code>FAILED_PRECONDITION</code>
(Opcional) si se implementa idempotencia con identificador del cliente y ya existe	<code>ALREADY_EXISTS</code>

El error debe incluir un mensaje claro y, cuando aplique, identificar el campo o regla que falló.

3.3.5 Evidencia por tags (obligatorio)

En el repositorio deben existir **dos tags**:

- **P1-LEGACY**: versión funcional mínima.
- **P1-REFACTOR**: versión final.

Ambos tags deben apuntar a commits distintos y estar disponibles en el repositorio entregado.

4. Entregables

Se entrega **un repositorio por práctica** en GitHub con nombre:

<NUM_GRUPO>_SoftwareAvanzado_Practica1

Ademas deben agregar el usuario del auxiliar como colaborador: **xhuniktzi**

Contenido obligatorio del repositorio:

Entregable	Ubicación sugerida	Condición
Servicio NestJS + TS con gRPC	<code>src/</code>	funcional
Protofile	<code>proto/quetzalship.proto</code>	completo y consistente
Colección Postman gRPC	<code>docs/postman/QuetzalShip.postman_collection.json</code>	ejecutable
Unit tests Jest	<code>test/</code> (o estructura equivalente)	<code>npm test</code>
README	<code>README.md</code>	instalación/ejecución/pruebas/supuestos/tags
Integrantes	<code>INTEGRANTES.md</code>	5 integrantes, nombre/carné/rol
Tags	Git	P1-LEGACY y P1-REFACTOR

5. Requerimientos mínimos

Para tener derecho a calificación, debe cumplirse todo lo siguiente:

1. El proyecto está implementado con **NestJS + TypeScript** y expone **solo gRPC**.

2. Implementa los **5 RPC** solicitados y cumple reglas de cálculo exactamente.
3. Maneja validaciones y devuelve errores con códigos gRPC consistentes.
4. Incluye **unit tests** ejecutables con `npm test`.
5. Incluye **proto/quetzalship.proto** y **colección Postman** en la ruta indicada.
6. Incluye **README.md** e **INTEGRANTES.md**.
7. Incluye tags **P1-LEGACY** y **P1-REFACTOR**.

6. Restricciones

- Persistencia **en memoria** (no base de datos).
- El modo de consumo oficial es **gRPC** (no REST).
- La solución debe poder ejecutarse en un entorno reproducible en Google Cloud (documentado en README).
- No se requiere autenticación/autorización.

7. Consideraciones adicionales

7.1 Historial de desarrollo

Se revisará el historial de commits para verificar avance incremental (no realizado en un único commit).

7.2 Cálculo de nota individual

La nota individual se obtiene a partir de la nota grupal y un índice de aporte calculado con Git Fame. Para efectos de este curso, **los equipos son siempre de 5**, por lo que el “aporte esperado” por integrante es **20** en una escala de 0 a 100.

Valores (0 a 100) tomados de Git Fame

- **LOC**: aporte por líneas de código
- **COMMITTS**: aporte por commits
- **FILES**: aporte por archivos modificados

Cálculos

- $\text{ÍNDICE} = 0.6 * \text{LOC} + 0.2 * \text{COMMITTS} + 0.2 * \text{FILES}$
- $\text{FACTOR} = \text{MIN}(\text{ÍNDICE} / 20, 1)$
- $\text{NOTA INDIVIDUAL} = \text{NOTA GRUPAL} * (0.6 + 0.4 * \text{FACTOR})$

Campo	Fórmula
ÍNDICE	$0.6 * \text{LOC} + 0.2 * \text{COMMITTS} + 0.2 * \text{FILES}$
FACTOR	$\text{MIN}(\text{ÍNDICE} / 20, 1)$

NOTA INDIVIDUAL	NOTA GRUPAL*(0.6 + 0.4*FACTOR)
-----------------	--------------------------------

7.3 Integridad académica

La copia total o parcial implica calificación de 0 y será reportada a la escuela de ciencias y sistemas.

8. Fecha y Modo de entrega

- **Fecha límite:** viernes 5 de diciembre, 23:59.
- **Entrega:** enlace del repositorio GitHub en la plataforma UEDi. El repositorio debe ser accesible e incluir todos los entregables y tags requeridos.